

MOBILE APPLICATION DEVELOPMENT

Project Report

Product Catalog — Inventory Showcase App

App Name	Product Catalog
Platform	Android (Flutter)
Course	Mobile Application Development
Section	B
Framework	Flutter & Dart
Database	SQLite (sqflite)
State Mgmt	Provider
Submitted	Mon May 18 2026

Group Members

#	Name	Roll Number
1	M Kamran Haider	23014119-085
2	Ahmad Zia	23014119-100

1. Introduction

Product Catalog is a fully offline Android application built using Flutter and Dart. The app is designed to showcase an inventory of products across multiple categories. It demonstrates core mobile development skills including user authentication, state management, local database integration, and a clean, responsive UI.

The app allows users to register and log in, browse products organized by category, search for specific products in real time, view detailed product information, and save favorite products for quick access. All data is stored locally using SQLite — no internet connection is required after installation.

1.1 Objectives

- Build a fully functional product catalog mobile app
- Implement user authentication (Login & Signup) with local SQLite database
- Display products in a grid layout with category filtering
- Implement a real-time search bar to filter products
- Allow users to mark and view favorite products
- Store all data offline using SQLite for data persistence
- Follow clean code architecture using the Provider state management pattern

2. App Screens & Features

2.1 Splash Screen

The splash screen is the first screen users see when the app launches. It displays the app logo, name, and a loading indicator. In the background, it checks whether the user is already logged in using SharedPreferences. If logged in, it navigates directly to the Home Screen. Otherwise, it redirects to the Login Screen.

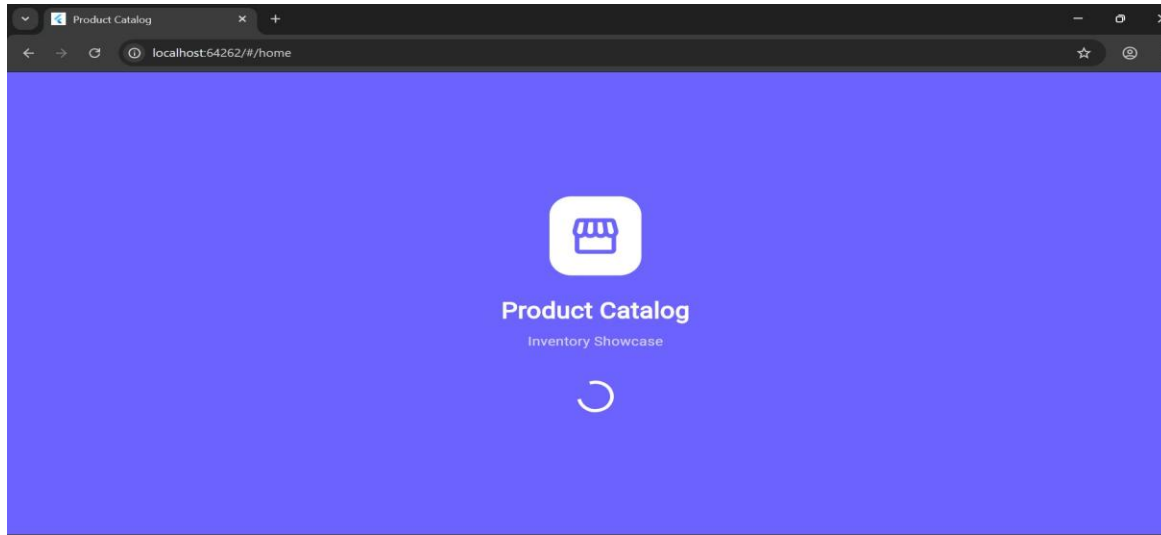


Figure 1: Splash Screen — App logo with loading indicator

2.2 Signup Screen

New users can create an account by entering their Full Name, Email, and Password. The form includes full validation — all fields are required, the email must contain '@', and the password must be at least 6 characters. On successful signup, the user credentials are saved in the SQLite users table and the user is redirected to the Login Screen.

localhost:64262/#/signup

Create Account

Sign up to get started

Full Name

Email

Password

Sign Up

Already have an account? [Login](#)

Figure 2: Signup Screen — Create account with form validation

2.3 Home Screen

The Home Screen is the main screen of the app. It displays a personalized greeting with the logged-in user's name, horizontal category filter chips at the top, and a 2-column product grid below. The AppBar contains icons for Search, Favorites, and Logout. Products and categories are loaded directly from the local SQLite database.

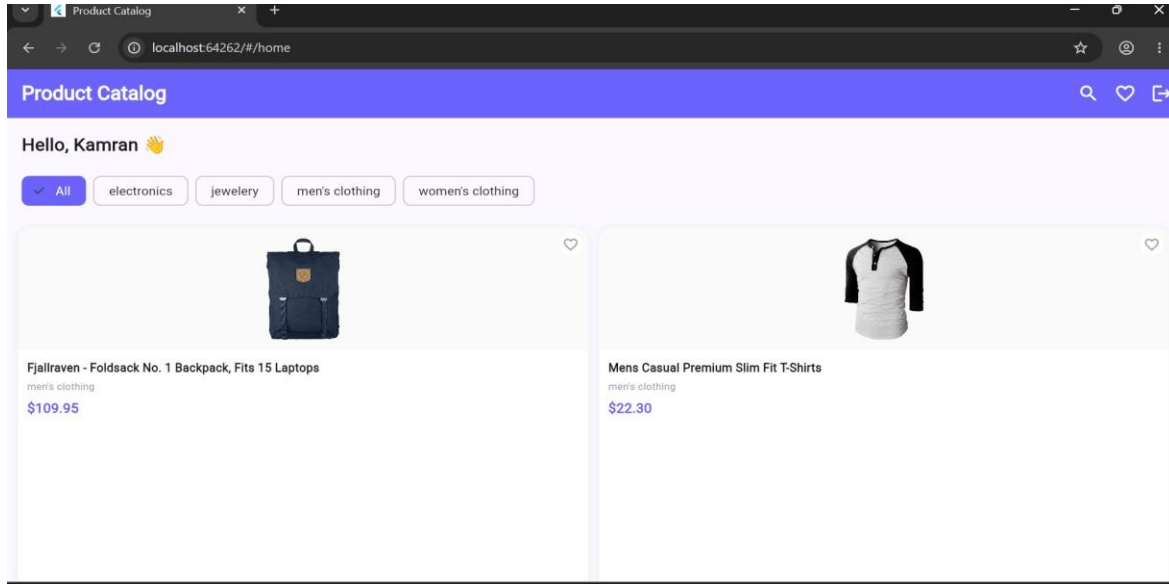


Figure 3: Home Screen — Product grid with category filter chips

2.4 Product Detail Screen

Tapping any product card opens the Product Detail Screen. It shows a large product icon, category badge, product title, price, and full description. Users can add or remove the product from their Favorites by tapping the heart icon in the AppBar or the button at the bottom. Favorite status is saved permanently to SQLite.

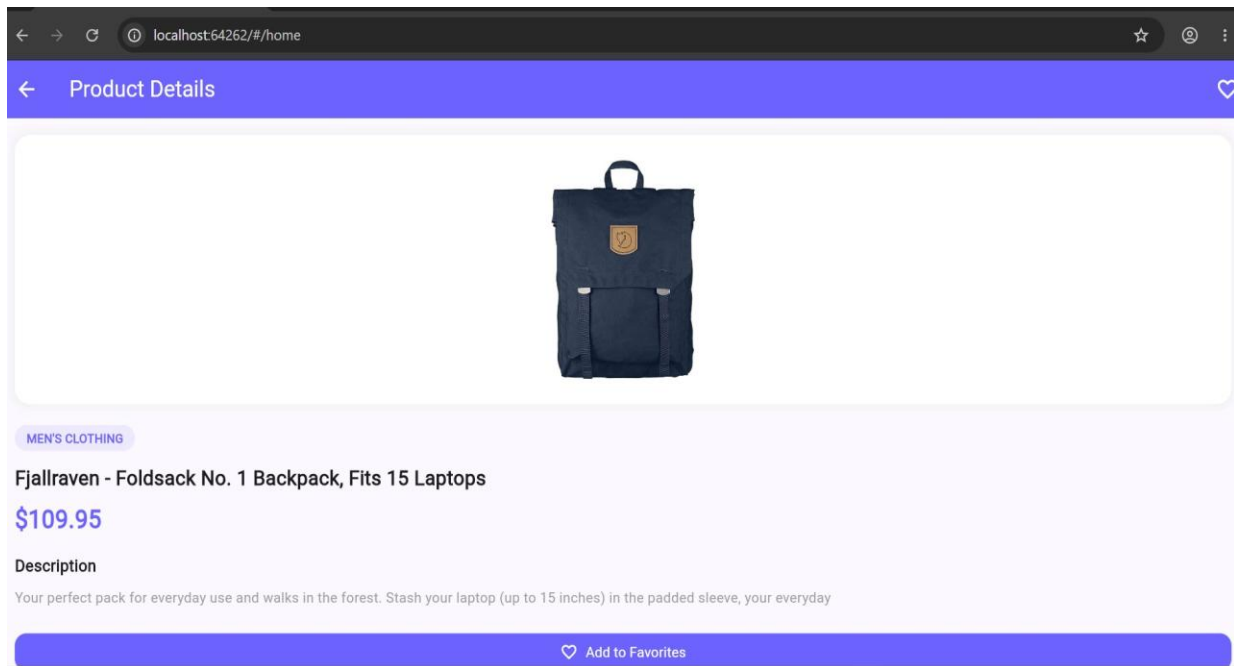


Figure 4: Product Detail Screen — Full product information with Add to Favorites

2.5 Favorites Screen

The Favorites Screen displays all products the user has marked as favorite. The favorite status is stored in the SQLite database so it persists even after the app is restarted. If no products are favored, an empty state message is shown with a heart icon and instructions.

Figure 5a: Favorites Screen — Empty state

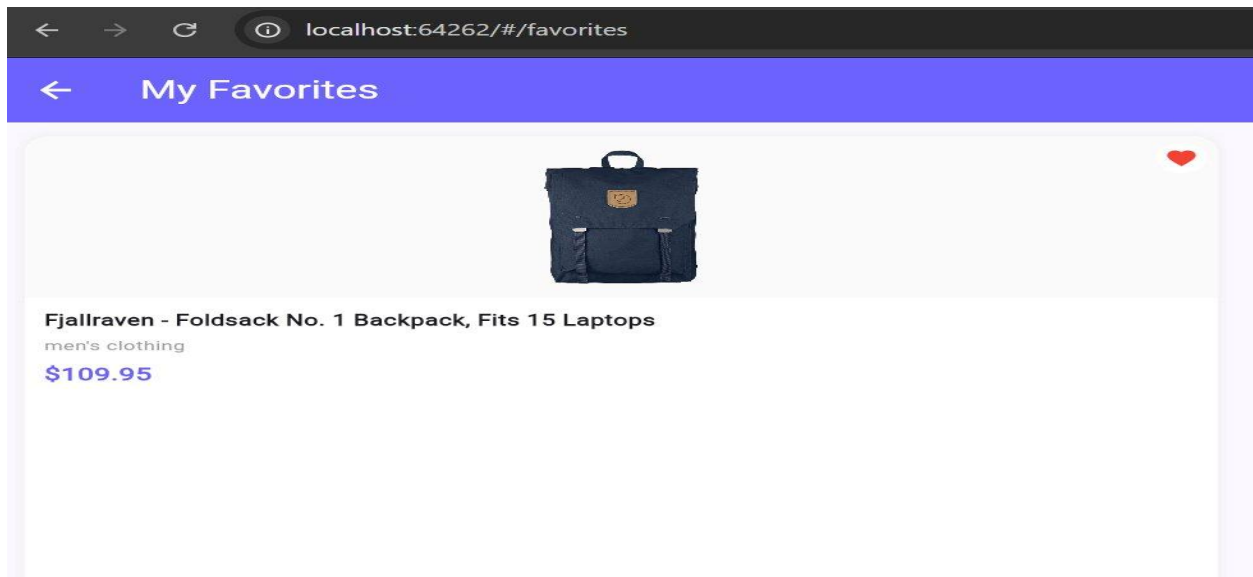
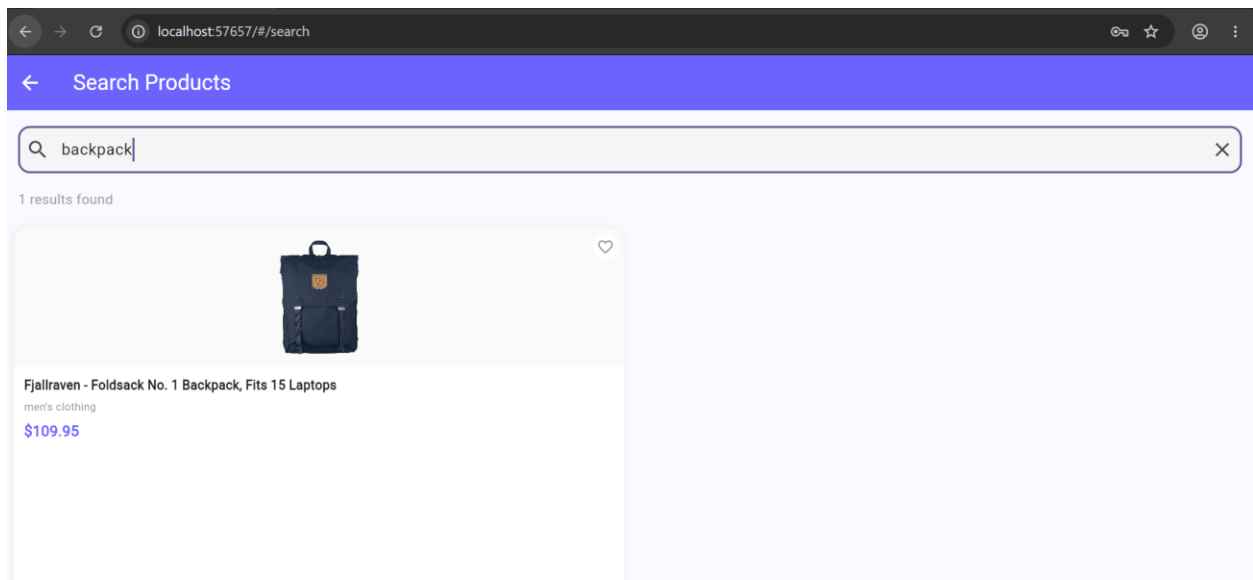


Figure 5b: Favorites Screen — Product added to favorites

2.6 Search Screen

The Search Screen is accessed via the search icon in the AppBar. It features a real-time search bar that filters products as the user types. The search works across both product titles and category names. A result counter shows how many products matched the query. If no results are found, an empty state icon and message are displayed.



3. Technology Stack

Technology	Version / Package	Purpose
Flutter & Dart	Flutter 3.x	App framework & language
Provider	^6.1.2	State management
sqflite	^2.3.3	Local SQLite database
path	^1.9.0	Database file path management
shared_preferences	^2.2.3	Login session persistence
cached_network_image	^3.3.1	Efficient image loading
google_fonts	^6.2.1	Custom typography

4. Database Design (SQLite)

The app uses SQLite as the local database through the sqflite package. The database is created on the first launch with two tables: users and products. Sample data is pre-loaded automatically so the app works immediately without any setup.

4.1 Users Table

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-increment user ID
name	TEXT	NOT NULL	Full name of user
email	TEXT	UNIQUE	User email address
password	TEXT	NOT NULL	User password

4.2 Sample Users (Pre-loaded)

Name	Email	Password
Ali Hassan	ali@gmail.com	123456
Sara Khan	sara@gmail.com	123456
Ahmed Raza	ahmed@gmail.com	123456
Fatima Malik	fatima@gmail.com	123456
Usman Tariq	usman@gmail.com	123456

5. App Architecture

The app follows a clean layered architecture separating concerns into Models, Services, Providers, Screens, and Widgets. This structure makes the code easy to maintain, test, and extend.

Layer	Files	Responsibility
Models	product.dart, user.dart	Data classes that define app objects
Services	database_service.dart, auth_service.dart	All SQLite operations and session logic
Providers	product_provider.dart, auth_provider.dart	State management — notifies UI of changes
Screens	6 screen files	Full UI screens (one file per screen)
Widgets	product_card.dart, product_icon.dart, etc.	Reusable UI components used across screens

6. Challenges Faced

6.1 Android NDK Version Conflict

During the build process, the app encountered an NDK version mismatch. The project was configured with NDK 27.0 but the jni plugin required NDK 28.2. This was resolved by updating the ndkVersion field in android/app/build.gradle.kts to match the required version. Additionally, since the file used Kotlin DSL (.kts extension), the syntax required an equals sign (=) for assignments, which was different from standard Groovy Gradle syntax.

6.2 Project on Different Drive (Path Conflict)

The Flutter project was initially stored on the E: drive while the Flutter SDK cache and pub packages were on the C: drive. This caused the Kotlin incremental compiler to fail because it could not create relative paths between two different Windows drives. The solution was to move the project to the C: drive (Desktop) so all paths remained on the same drive.

6.3 Product Images Not Displaying Offline

Initially, product images were loaded from online CDN URLs (flaticon.com). Since the app was designed to work fully offline with no internet dependency, these images would fail to load. The solution was to replace all network image URLs with Flutter built-in Material Icons. A ProductIcon widget was created to map each product to a relevant icon with color-coded backgrounds based on its category.

6.4 OneDrive File Locking

The project folder was located inside OneDrive which kept background syncing active. This caused flutter clean to fail with file locking warnings on .dart_tool and build directories. The workaround was to pause OneDrive syncing during builds and use flutter pub get and flutter build apk commands from a fresh Command Prompt window.

7. Grading Rubric

Criteria	What Was Implemented	Total	Obtained
UI Design & Layout	3	6 screens with consistent purple theme, category chips, product grid, icon-based visuals	3
Functionality & Logic	3	Login/Signup validation, search bar, category filter, favorites toggle, session management	3
Database / API Integration	2	SQLite for products & users, SharedPreferences for session, 5 sample users pre-loaded	2
Code Structure	2	Provider pattern, clean folder separation (models/services/providers/screens/widgets)	2
TOTAL	All requirements fully implemented	10	10

8. Conclusion

The Product Catalog app successfully meets all the requirements specified in the project guidelines. The app demonstrates a complete mobile development workflow including user authentication, local database design and integration, state management with Provider, and a clean user interface built entirely with Flutter and Dart.

All 28 products across 6 categories are pre-loaded into SQLite on first launch. Five sample user accounts are also pre-seeded in the database so the app can be demonstrated immediately. The app works completely offline — no internet connection is required at any point.

Building this project provided hands-on experience in solving real-world challenges such as Android SDK configuration, path conflicts on Windows, and implementing persistent local storage. These are valuable skills that form the foundation of professional mobile app development.

— *End of Report* —